Smart-X IoT Mesh Ecosystem

Final PoE Report: Network Topology and Mesh Routing (Part 3)

Nosipho Buhlebemvelo Swazi Kubheka
ST10083146 – Emeris Durban North Campus

Contents

1. Introduction 2
2. Implementation Report: Network Topology Feature 3
   2.1 How the pieces fit together 3
   2.2 AVL tree: historical telemetry queries 3
   2.3 The mesh as a graph 6
   2.4 Graph traversal: BFS and DFS 6
   2.5 Min-heap and Prim's minimum spanning tree 7
   2.6 Routing and self-healing 9
   2.7 Complexity summary 10
3. Project Completion Report 11
   3.1 What was delivered 11
   3.2 Challenges and how they were overcome 11
   3.3 Key learnings 12
   3.4 Known limitations 12
4. Technology Recommendations and Rationale 13
   4.1 Justification 13

5.AI Use Declaration 15

References 16

1. Introduction

The Smart-X system is a gateway for a mesh of ESP32 sensors and actuators. It has three pillars, and all three now run inside one dashboard: (1) Sensor Data Ingestion and Telemetry from Part 1, (2) the Real-Time Command Stream and History from Part 2, and (3) Network Topology and Mesh Routing, built for this final part. The backend is an ASP.NET Core 10 Minimal API. The frontend is React with Vite. SignalR pushes live updates to the browser.

Part 3 adds the data structures the brief asks for: an AVL tree for historical telemetry queries, a graph of the physical mesh, Breadth-First and Depth-First Search to see which devices are still connected, a hand-built min-heap, and Prim's minimum spanning tree (MST) for power-efficient routing. Section 2 explains each one with code from the project. Section 3 is the project completion report, and Section 4 gives technology recommendations.

2. Implementation Report: Network Topology Feature

2.1 How the pieces fit together

Every reading that a device sends to POST /api/telemetry is processed by a background worker (Part 2). In Part 3 the worker also stores each reading in a per-sensor AVL tree. When a device is registered it also becomes a vertex in the mesh graph. The Network Topology page then reads three kinds of answer from the API:

- **The graph:** nodes, links, which links are in the MST, and who is down or cut off (GET /api/mesh/topology).
- **Routes and traversals:** the path from any MAC address or node id to the gateway, and BFS or DFS lists of connected devices (GET /api/mesh/route/{id}, GET /api/mesh/connected).
- **History:** all readings of one sensor between two timestamps, answered by the AVL tree (GET /api/mesh/nodes/{id}/history).

2.2 AVL tree: historical telemetry queries

Problem. The brief asks for queries such as all moisture readings between 14:00 and 16:00 for Node A without iterating through a massive, unorganised list. A sensor reporting every two seconds produces over 40,000 readings a day, so scanning every reading for each question gets slower and slower.

Why an AVL tree. A binary search tree keyed by timestamp can jump straight to the start of a time range. A plain BST is risky for telemetry, because timestamps arrive in increasing order and the tree would degrade into a linked list of height n. An AVL tree (Adelson-Velsky and Landis, 1962) avoids this. Every node stores its height, and its balance factor (left height minus right height) must stay between -1 and +1. When an insert breaks that rule, the tree rotates itself back into balance (Cormen et al., 2022). The sorted dictionary used in Part 2 keeps readings in order but has no way to start at a given key, so it cannot answer a range question without walking from the beginning.

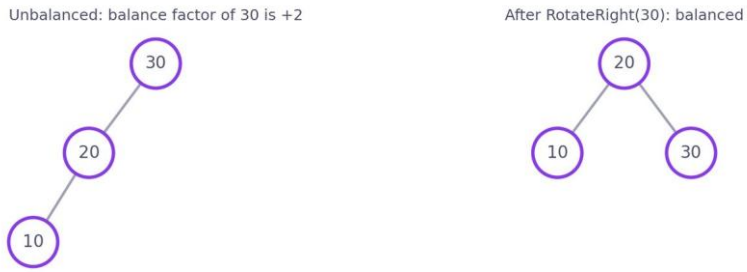


Figure 1: A right rotation fixes a node whose left side is two levels taller than its right side.

The code below is the rebalancing step and the rotation. The same four cases (left-left, left-right, rightright, right-left) are handled by one or two rotations.

```
private Node Rebalance(Node node)
{
    var balance = BalanceFactor(node);

    if (balance > 1) // left side too heavy
    {
        // Left-Right case: turn it into a Left-Left case first
if (BalanceFactor(node.Left!) < 0)
        {
            node.Left = RotateLeft(node.Left!);
        }
```

```
        return RotateRight(node);
    }

    if (balance < -1) // right side too heavy
    {
        // Right-Left case: turn it into a Right-Right case first
if (BalanceFactor(node.Right!) > 0)
        {
            node.Right = RotateRight(node.Right!);
        }
        return RotateLeft(node);
    }

    return node; // already balanced
}
```

```
//        y                x
//       / \              / \
//      x   C    --->    A   y
//     / \                  / \
//    A   B                B   C
private Node RotateRight(Node y)
{
    var x = y.Left!;
    y.Left = x.Right;
    x.Right = y;
    UpdateHeight(y); // y is now lower, so update it first
    UpdateHeight(x);
return x;
}
```

Range query. The query visits a node, goes left only if the start of the range is smaller than the node's key, adds the node if it is inside the range, and goes right only if the end of the range is bigger than the node's key. Whole branches that cannot hold a match are never entered, so the cost is about O(log n + k), where k is the number of readings returned. The method also counts the nodes it touched, so the dashboard can show how little of the tree was used.

```
private void CollectRange(Node? node, TKey low, TKey high, List<TValue> results, ref int
visited)
{
    if (node == null) return;
visited++;

    var compareLow = low.CompareTo(node.Key);   // < 0 means low is smaller than this key
var compareHigh = high.CompareTo(node.Key); // > 0 means high is bigger than this key
    // Only go left if there could be keys smaller than this one that are still >= low.
if (compareLow < 0)
    {
        CollectRange(node.Left, low, high, results, ref visited);
    }

    if (compareLow <= 0 && compareHigh >= 0)
    {
        results.Add(node.Value);
    }

    // Only go right if there could be keys bigger than this one that are still <= high.
if (compareHigh > 0)
    {
        CollectRange(node.Right, low, high, results, ref visited);
    }
}
```

How the archive uses it. TelemetryArchive keeps one AvlTree per sensor, with the timestamp in ticks as the key. Keys must be unique, so if two readings share a timestamp the second key is nudged by one tick. A lock protects the tree because the background worker writes while API requests read.

```
public void Add(string mac, TelemetryPacket<float> packet)
{
    lock (_lock)
    {
        if (!_trees.TryGetValue(mac, out var tree))
        {
            tree = new AvlTree<long, TelemetryPacket<float>>();
            _trees[mac] = tree;
        }

        // keys must be unique, so nudge by one tick if two
readings share a timestamp         var key =
packet.Timestamp.UtcTicks;         while (tree.TryGet(key, out
_))
        {             key++;
        }
        tree.Insert(key, packet);
    }
}
```

Measured result. The dashboard has a benchmark button. It builds a fake history in random insertion order, then asks for the same 2-hour window with the AVL tree and with a plain list scan. Table 1 shows one run on the development build. Absolute times depend on the computer, but the trend does not: the AVL query stays near 250 microseconds, because it mostly returns the ~7,200 matching readings, while the list scan grows with the size of the history. In every run the tree was verified as a valid AVL tree and returned the same readings as the list.

History size	AVL query	List scan	Nodes visited by AVL	Speed-up	Tree height

50,000	287.4 µs	1,205.3 µs	7,215	4.2×	19
200,000	224.3 µs	4,526.6 µs	7,221	20.2×	21
500,000	250.8 µs	11,137.4 µs	7,221	44.4×	23

Table 1: AVL range query against a plain list scan (one run; the window returned about 7,200 readings each time).

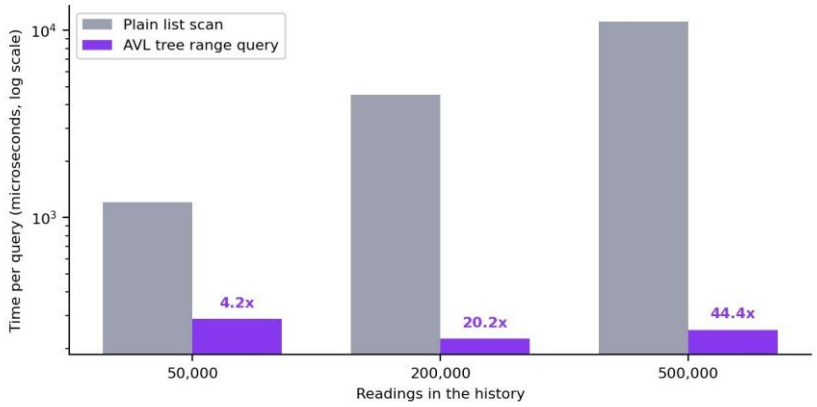


Figure 2: Query time on a log scale. The AVL tree height grows by only 4 levels while the history grows 10 times.

2.3 The mesh as a graph

The physical network is modelled as an undirected weighted graph. Each registered ESP32 is a vertex, and there is one extra vertex for the central gateway. Each wireless link is an edge whose weight is the power needed to send a packet over it, so a lower number is better. The graph is stored as an adjacency list, a Dictionary from each node to the list of links leaving it. A wireless link works in both directions, so each link is stored on both of its nodes.

```
private void ConnectUnlocked(string a, string b, double cost)
{
    // a wireless link works both ways, so the edge is stored on
both nodes (undirected graph)
    _graph[a].Add(new MeshEdge(b, cost));
    _graph[b].Add(new MeshEdge(a, cost));
}
```

In a real deployment the links would be discovered from radio signal strength. For this simulation they are generated when a device registers: it links to its two cheapest neighbours (the same zone is cheaper than a different zone), some devices get a third link chosen from a stable hash of the MAC address so the result is the same every run, and the first device in each zone also gets an uplink to the gateway. Users can add or remove links on the dashboard. A node is treated as down if it was failed from the dashboard or the Part 2 watchdog marked it disconnected. Every algorithm receives an isUp function and skips down nodes, which is how the network heals around a failure.

2.4 Graph traversal: BFS and DFS

Breadth-First Search uses a Queue and visits the gateway, then all its neighbours, then their neighbours. Because it works level by level, the level number is the smallest number of wireless hops from the gateway to each device. Depth-First Search uses a Stack and follows one branch as far as it can before backing up. I wrote it with an explicit stack instead of recursion, so a long chain of nodes cannot overflow the call stack.

```
public static List<(string Id, int Hops)> Bfs(Dictionary<string, List<MeshEdge>> graph,
string start, Func<string, bool> isUp)
{
    var order = new List<(string Id, int Hops)>();
    if (!graph.ContainsKey(start) || !isUp(start)) return order;

    var visited = new HashSet<string> { start };
var queue = new Queue<(string Id, int Hops)>();
queue.Enqueue((start, 0));

    while (queue.Count > 0)
    {
        var current = queue.Dequeue();
order.Add(current);

        foreach (var edge in graph[current.Id])
        {
            if (!isUp(edge.To)) continue;
            if (!visited.Add(edge.To)) continue; // Add returns false if it was already
visited

            queue.Enqueue((edge.To, current.Hops + 1));
        }
    }

    return order;
}
```

Both run in O(V + E). They answer the brief's connectivity question in three ways. First, the dashboard lists every device currently connected to the gateway with its hop count. Second, a zone filter acts as the subnet: the search may pass through other zones to reach the gateway, but only devices from the chosen zone are listed. Third, comparing the nodes that are up with the nodes the search reached exposes devices that are cut off from the gateway. These are shown in red.

2.5 Min-heap and Prim's minimum spanning tree

A minimum spanning tree connects every node using the smallest possible total link cost. For the mesh, that is the cheapest set of links to keep switched on so that every device can still reach the gateway. Prim's algorithm (Prim, 1957) starts at the gateway and keeps adding the cheapest link that joins a node already in the tree to one that is not. It needs a fast way to get the cheapest link each time, which is exactly what a heap provides.

Part 2 used the built-in PriorityQueue (Microsoft, n.d.a). For Part 3 the brief names heaps as a topic, so I wrote a binary min-heap myself. It is stored in a list, the children of index i are at 2i+1 and 2i+2, and Push and Pop each move an item along a single path, so both are O(log n) (GeeksforGeeks, n.d.c).

```
private void SiftDown(int index)
{
    var count = _data.Count;
```

```
    while (true)
    {
        var left = 2 * index +
1;         var right = left + 1;
var smallest = index;

        if (left < count && _data[left].Priority < _data[smallest].Priority) smallest =
left;
        if (right < count && _data[right].Priority < _data[smallest].Priority) smallest =
right;

        if (smallest == index) break;

        Swap(index, smallest);
index = smallest;
    }
}
public static MstResult Prim(Dictionary<string, List<MeshEdge>> graph, string root,
Func<string, bool> isUp)
{
    var result = new MstResult();
    if (!graph.ContainsKey(root) || !isUp(root)) return result;

    var heap = new MinHeap<MstEdge>();
result.Included.Add(root);

    foreach (var edge in graph[root])
    {
        if (isUp(edge.To))
        {
            heap.Push(new MstEdge(root, edge.To, edge.Cost), edge.Cost);
        }
    }

    while (heap.Count > 0)
    {
        var cheapest = heap.Pop().Item;

        // the heap can still hold old links to nodes we have since reached another way
if (result.Included.Contains(cheapest.To)) continue;

        result.Included.Add(cheapest.To);
result.Edges.Add(cheapest);
        result.Parent[cheapest.To] = cheapest.From;
result.TotalCost += cheapest.Cost;

        foreach (var edge in graph[cheapest.To])
        {
            if (isUp(edge.To) && !result.Included.Contains(edge.To))
            {
                heap.Push(new MstEdge(cheapest.To, edge.To, edge.Cost), edge.Cost);
            }
        }
    }
```

```
    return result;
}
```

With a heap, Prim's algorithm runs in O(E log E). Stale heap entries (links to nodes already reached another way) are simply skipped when they come out. If some nodes cannot be reached, the loop ends without them and the dashboard reports them as cut off. This is the algorithm described by GeeksforGeeks (n.d.b), implemented here in C# with my own heap.

Result. For the six demo devices (13 links), the MST keeps 6 links at a total power cost of 82. If every link were used the cost would be 231, so the tree needs about 65% less power. Figure 3(a) shows the tree in green.

2.6 Routing and self-healing

A node's route to the gateway is its path up the MST. When a node fails, the MST is recomputed without it, and the routes before and after are compared. The report lists every node whose route changed, with its new next hop, which answers the brief's question of where to hop when a device is down. In Figure 3(b) moisture-sensor-a fails and five nodes find new routes. For example, power-meter1 used to go through moisture-sensor-a, and now goes through moisture-sensor-b, pump-a and powermeter-2 to the gateway. The total cost drops from 82 to 78, because the failed node's links no longer need to be kept on. Four routes even get cheaper (for example pump-a goes from 50 to 29), because the Zone 2 devices can no longer take the cheap link through moisture-sensor-a and must use powermeter-2's own uplink, which shows again that the MST optimises the whole network rather than each path. In Figure 3(c) the second zone uplink also fails, and the four remaining devices have no path left, so they are reported as cut off.

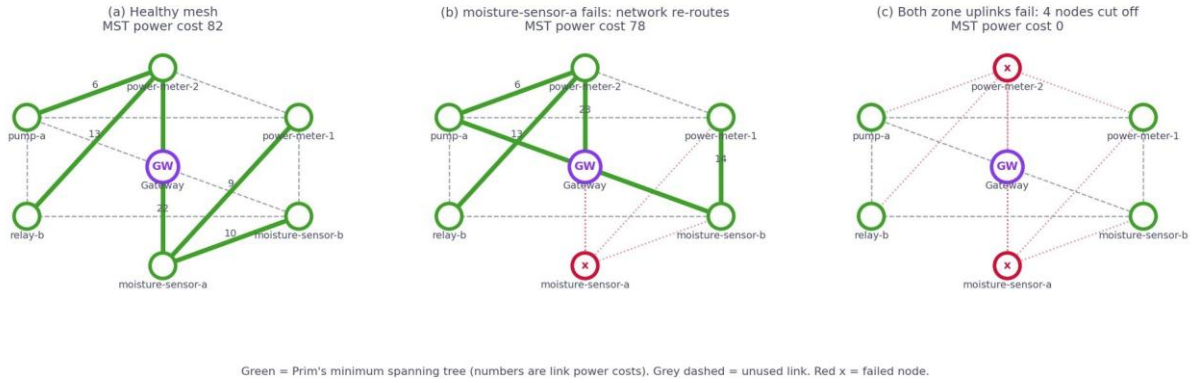


Figure 3: Real topologies taken from the running API. (a) healthy, (b) one node failed, (c) both uplinks failed.

MST route versus shortest path. While testing I noticed that a node's route along the MST can cost more than its own cheapest possible path. For pump-a the MST route costs 50 over 3 hops, while the cheapest single path costs 29 over 2 hops. This is not a bug. The MST minimises the cost of the whole backbone, not the cost for one node. To make this clear, the route panel also shows Dijkstra's shortest path (Dijkstra, 1959), which uses the same heap. The tree answers the question "what is the cheapest network to keep running?" and Dijkstra answers "what is the cheapest path for this one node?"

2.7 Complexity summary

Structure or algorithm	Where it is used	Cost
AVL tree	Per-sensor telemetry history; range queries	Insert and find O(log n); range query O(log n + k)
Adjacency list	Mesh graph (devices and links)	Space O(V + E)
BFS / DFS	Connected devices, hop counts, cutoff detection	O(V + E)
Min-heap (own implementation)	Priority of links in Prim's and Dijkstra	Push and Pop O(log n)
Prim's MST	Power-efficient backbone and routes	O(E log E)
Dijkstra	Cheapest single path, shown for comparison	O(E log V)

Table 2: Data structures and algorithms added in Part 3.

The README explains how to compile, run and test every feature, including step-by-step checks for the AVL range queries, the benchmark, route tracing by MAC address, BFS and DFS, node failure and recovery, and editing links.

3. Project Completion Report

3.1 What was delivered

Part	Pillar	Main techniques
1	Sensor Data Ingestion and Telemetry	Generic TelemetryPacket<T>, operator overloading on power readings, jagged arrays into List<T>, recursive validation of the Facility, Zone, Sub-Zone and Node tree, file upload, live feed over SignalR.
2	Real-Time Command Stream and History	Queue and priority queue for telemetry, Stack for command undo, Dictionary keyed by MAC, SortedDictionary for history, HashSet for active faults, pattern-learning suggestions and early warnings, simulated devices that call the API over HTTP.
3	Network Topology and Mesh Routing	AVL tree range queries, mesh graph, BFS and DFS, own minheap, Prim's MST, Dijkstra comparison, failure simulation with rerouting, route tracking by MAC address, historical log and benchmark.

Table 3: The three parts of the project.

3.2 Challenges and how they were overcome

Connecting the C# backend to the React frontend. In Part 1 the registration form sent the sensor category as a string, but ASP.NET Core expects an enum as a number by default, so every registration came back as a 400 error with a vague message. The cause was found by reading the error shape and comparing it with what the frontend sent. The fix was to register a JSON string-enum converter for both the REST API and the SignalR protocol, so every enum travels as readable text on both channels.

Keeping live data when the user changes page. In Part 1 each page opened its own SignalR connection, so the live feed vanished whenever the user navigated. In Part 2 I moved the connection and the device state into a React Context that sits above the router. In Part 3 the same context raises a meshChanged signal, so the topology page redraws as soon as a node fails, without losing anything when the user switches pages.

Making simulated data behave like hardware. The brief says simulated data must hit the API as a device would. The first idea of writing readings straight into memory would have hidden bugs in the API. Instead, a background service sends real HTTP POST requests, and actuators poll the API for the state the operator wants. This also meant the simulated mesh could be tested against failures, such as an unplugged device.

Shared structures and several threads. The background worker, the simulator and the API endpoints all touch the same Dictionary, queues, SortedDictionary, HashSet and AVL trees, and none of these classes is thread-safe. Each wrapper class takes a lock around every operation, and API endpoints only receive copies of the data, so a request can never see a half-updated structure.

Generating a believable mesh. My first link generator connected every device straight to the gateway. The code worked, but BFS showed every node one hop away and the MST was trivial, so failures changed nothing. I redesigned it so devices link to nearby neighbours and only the first device in each zone gets a gateway uplink. Re-running the tests then showed multi-hop routes and real rerouting.

MST versus shortest path. Comparing routes showed that the MST route for a node can be longer than its cheapest path. Rather than hiding this, I added Dijkstra's algorithm next to the MST on the dashboard and explained the difference (Section 2.6).

Unique keys in the AVL tree. Two readings can carry exactly the same timestamp, and an AVL tree keyed by timestamp would overwrite one of them. The archive checks for an existing key and moves the new one forward by one tick.

3.3 Key learnings

- **Choose the structure from the question being asked.** Part 2's questions (what is next, what is urgent, what was the last command, which device has this MAC address, which faults are active) each matched a different collection. In Part 3 the benchmark showed the cost of the wrong choice: for a time-window question, a plain list was 44 times slower than an AVL tree at 500,000 readings.
- **Graph algorithms answer different questions.** BFS and DFS tell me who is reachable, Prim's tells me the cheapest backbone, and Dijkstra tells me the cheapest path for one node. Using the wrong one gives a correct-looking but wrong answer.
- **Failure is part of the design.** Making every algorithm skip nodes that are down, instead of adding failure handling afterwards, is what made the self-healing report possible.
- **Test by running, not by reading.** Most problems (the enum error, the trivial mesh, the MST cost surprise) were found by running the API and sending real requests. The final check is always a clean clone that builds and runs, because the brief awards no marks for code that does not compile and run.
- **AI assistance needs verification.** AI helped me write and explain much of the code, but its output only counted once it had been compiled, run and tested.

3.4 Known limitations

All data is held in memory, so restarting the API resets devices, links and history. Mesh links are generated rather than measured from radio signal strength. The AVL tree supports insert and query but not delete, so history is never removed. A real deployment would need persistence, real device connectivity and authentication, which is what the recommendations below address.

4. Technology Recommendations and Rationale

The recommendations below are ordered as a roadmap. Each one is chosen because it fits the existing .NET and React ecosystem, so none of them requires rewriting what already works.

Recommendation	What it adds	Why it suits Smart-X and .NET
1. CI/CD with GitHub Actions	Builds and tests the API and the React client on every push.	The repository is already on GitHub, and GitHub documents a standard workflow for building and testing .NET (GitHub, n.d.). It would catch the "does not compile and run" failure before submission.
2. Persistent storage with PostgreSQL	Keeps devices, links and telemetry across restarts.	Npgsql provides an Entity Framework Core provider for PostgreSQL (Npgsql, n.d.), so the data layer stays in C#. The AVL tree remains as a fast in-memory index over the stored history.
3. Azure IoT Hub for real devices	Replaces the simulator with per-device identity and secure two-way messaging.	IoT Hub is a managed service for communicating with large numbers of devices (Microsoft, n.d.b). Each MAC address would become a device identity, and the existing telemetry endpoint logic can be reused behind it.
4. LLM summaries of daily alerts	Turns the day's alerts and faults into a short plainlanguage report.	Microsoft.Extensions.AI gives .NET one chat interface that can sit on top of different model providers (Microsoft, n.d.c; Microsoft, n.d.d), so the model can be changed without changing the application.
5. Home Assistant integration	Shows Smart-X devices in a smart-home dashboard and allows home automations.	Home Assistant (n.d.) is an open home automation platform. It would connect to Smart-X through a message broker rather than .NET code, so it suits home deployments best.

Table 4: Recommended technologies.

4.1 Justification

GitHub Actions (security, quality). A workflow that runs dotnet build and npm run build on every push gives an automatic compile check. This directly protects against the biggest risk in this project, code that no longer runs. A minimal workflow would look like this:

```
name: ci
on: [push,
pull_request] jobs:
build:
    runs-on: ubuntu-latest
steps:
-      uses: actions/checkout@v4
-      uses: actions/setup-dotnet@v4        with:
          dotnet-version: 10.0.x
-      run: dotnet build SmartX.Api
-      uses: actions/setup-node@v4        with:
          node-version: 20
-      run: npm ci && npm run build        working-
directory: smartx-client
```

(This workflow is proposed, not part of the submitted project.)

PostgreSQL with Npgsql (scalability, reliability). Today a restart erases the whole history. Storing readings in a database fixes that, and a managed PostgreSQL service from a cloud provider would add backups and replication. Because the provider plugs into Entity Framework Core, the change is mostly a new data-access layer, not a redesign.

Azure IoT Hub (security, scalability). The current simulator trusts any caller of /api/telemetry that knows a MAC address. A real fleet needs each device to authenticate. IoT Hub provides per-device identities and a managed service that scales to many devices, and it supports the messaging style that ESP32 boards already use.

LLM summaries (workflow). Operators currently read alerts one by one. A scheduled job could send only the day's alert text (no personal data) to a model through the IChatClient abstraction and store the short summary on the dashboard. The risks are cost, occasional wrong summaries and data privacy, so the summary should be labelled as machine-written and the raw alerts must stay one click away.

Home Assistant (reach). For smart-home use, exposing the sensors and relays to Home Assistant would allow automations such as switching a pump on when the moisture drops, using the same data Smart-X already collects. It adds value for home users but is optional for industrial deployments.

5.AI Use Declaration

I used an AI assistant (Claude, made by Anthropic) while building this project. It helped me plan the architecture, and explain source code, find and fix build and runtime errors, and draft this report and the README. I ran, tested and reviewed the code before submitting it. The sources below were used to understand the algorithms and tools; no code was copied from them.

References

Adelson-Velsky, G.M. and Landis, E.M. (1962) 'An algorithm for the organization of information', Doklady Akademii Nauk SSSR, 146, pp. 263–266.

Cormen, T.H., Leiserson, C.E., Rivest, R.L. and Stein, C. (2022) Introduction to Algorithms. 4th edn. Cambridge, MA: MIT Press.

Dijkstra, E.W. (1959) 'A note on two problems in connexion with graphs', Numerische Mathematik, 1(1), pp. 269– 271.

GeeksforGeeks (n.d.a) AVL Tree | Set 1 (Insertion). Available at: https://www.geeksforgeeks.org/avl-tree-set-1insertion/ (Accessed: 29 October 2026).

GeeksforGeeks (n.d.b) Prim's algorithm using priority_queue in STL. Available at: https://www.geeksforgeeks.org/prims-algorithm-using-priority_queue-stl/ (Accessed: 29 October 2026).

GeeksforGeeks (n.d.c) Types of Heap Data Structure. Available at: https://www.geeksforgeeks.org/types-of-heapdata-structure (Accessed: 29 October 2026).

GitHub (n.d.) Building and testing .NET. Available at: https://docs.github.com/en/actions/tutorials/build-and-testcode/net (Accessed: 29 October 2026).

Home Assistant (n.d.) Home Assistant. Available at: https://www.home-assistant.io/ (Accessed: 29 October 2026).

Microsoft (n.d.a) PriorityQueue<TElement,TPriority> Class. Available at: https://learn.microsoft.com/dotnet/api/system.collections.generic.priorityqueue-2 (Accessed: 29 October 2026).

Microsoft (n.d.b) Azure IoT Hub concepts overview. Available at: https://learn.microsoft.com/azure/iot-hub/iot-hubdevguide (Accessed: 29 October 2026).

Microsoft (n.d.c) Microsoft.Extensions.AI libraries. Available at: https://learn.microsoft.com/dotnet/ai/ai-extensions (Accessed: 29 October 2026).

Microsoft (n.d.d) Use the IChatClient interface. Available at: https://learn.microsoft.com/dotnet/ai/ichatclient (Accessed: 29 October 2026).

Npgsql (n.d.) Npgsql Entity Framework Core Provider for PostgreSQL. Available at: https://npgsql.org/efcore (Accessed: 29 October 2026).

Prim, R.C. (1957) 'Shortest connection networks and some generalizations', Bell System Technical Journal, 36(6), pp. 1389–1401.